

Chapter 1

Introduction

OSEK/VDX [34][43] is a standard of automobile OS, developed by a consortium of European automobile manufacturers and suppliers in conjunction with the University of Karlsruhe in 1994. The primary motivation of OSEK/VDX standard is to resolve the problem of increasing software content in vehicles and desire high-quality products. Currently, it has been widely adopted by many automobile manufacturers to design and develop a vehicle-mounted OS, such as BMW, Opel and TOYOTA. Based on the OSEK/VDX OS, more and more applications are developed and deployed in vehicles to assist drivers to control vehicles, such as the cruise control system and temperature control system. To an application which runs on the OSEK/VDX OS, the reliability is very important, since a nonreliance application will threaten our life. However, with the growing functionalities in vehicles and increasing complexity in the development, how to straitly ensure the reliability of the developed OSEK/VDX applications is becoming a challenge for developers.

1.1 OSEK/VDX Applications and Model Checking

An application developed on OSEK/VDX OS consists of multiple tasks (OSEK/VDX application is multi-tasks software). To concurrently execute multiple tasks on one CPU, the *static priority scheduling policy* with non-preemptive strategy and full-preemptive strategy is adopted by OSEK/VDX scheduler to dispatch tasks. Particularly, as shown in Fig. 1.1,

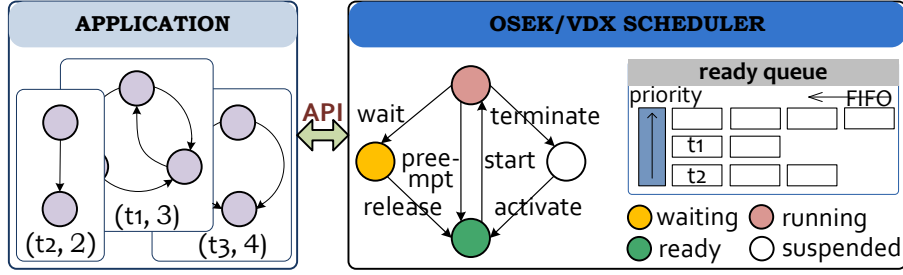


Figure 1.1: OSEK/VDX scheduler

a ready queue is used to lay out the execution order of tasks. Based on the ready queue, the running task is explicitly determined by scheduler (deterministic scheduler is used to dispatch tasks). OSEK/VDX OS can process two types of tasks, basic task and extended task. The states of a basic task consist of *running* state, *ready* state and *suspended* state. Compared with the basic task, the extended task can hold synchronization events and has a unique state called *waiting* state. Furthermore, OSEK/VDX OS provides a standardized application interfaces (APIs) for its applications (such as *ActivateTask(tk)*, *TerminateTask()*, *WaitEvent(eid)* and *GetResource(rid)*), and tasks within the application can invoke these APIs to dynamically change the states of tasks, synchronization events and shared resources, e.g., invoke the API *ActivateTask(tk)* to activate a suspended task.

To ensure the reliability of OSEK/VDX applications, testing as an important checking technique is being used in industry. However, there are several disadvantages when we apply testing technique to check the developed OSEK/VDX applications, e.g., as we know, it is difficult to exhaustively check an OSEK/VDX application using testing technique, especially when the tested OSEK/VDX application holds non-deterministic behaviours, because of the incomplete test-cases. In addition, it is hard to reproduce an error trace when we detect the error using testing technique, due to the concurrency of OSEK/VDX applications.

Model checking [2][3], an efficient and exhaustive verification technique for concurrent software, can be applied to verify OSEK/VDX applications instead of testing technique. In contrast with testing technique, there are several advantages in model checking technique,

e.g., it is usually an exhaustive and automatic checking technique. Particularly, it will return a counterexample when detecting an error. Based on the returned counterexample, developers can easily find the error trace and reproduce the erroneous behaviours.

1.2 Shortcomings of Existing Model Checking for OSEK/VDX Applications

There exist many model checking methods [26][50][51] that have been applied to verify concurrent software such as the ANSI-C multi-threaded software and SystemC multi-threaded software. In the existing methods, due to the concurrency of target software, all of the interleavings of threads are checked in the verification process in order to exhaustively verify the given software. Although the existing methods can exhaustively check concurrent software, these methods usually perform an approximate verification since the behaviours of scheduler are not taken into account in verification process. If we apply these existing model checking methods to verify OSEK/VDX applications, it is too imprecise since a lot of unnecessary interleavings of tasks will be superfluously checked in the verification stage, especially these unnecessary interleavings will usually result in a spurious bug. Furthermore, as a result of the spurious bug, developers have to spend extra costs judging whether the detected bug is real one or not after completing verification. As to reduce the checking costs, a more accurate model checking approach should be used in the verification of OSEK/VDX applications.

Recently, to accurately check concurrent software such as ANSI-C multi-threaded software and SystemC multi-threaded software, several prominent model checking methods [13][37] have been proposed by some senior researchers. In these methods, as to seek a more accurate verification result, the behaviours of scheduler are taken into account in the verification process. Even so, these prominent model checking methods are still unsuitable to accurately verify OSEK/VDX applications because of different scheduling policy. In the existing methods, the running thread within the concurrent software is arbitrarily

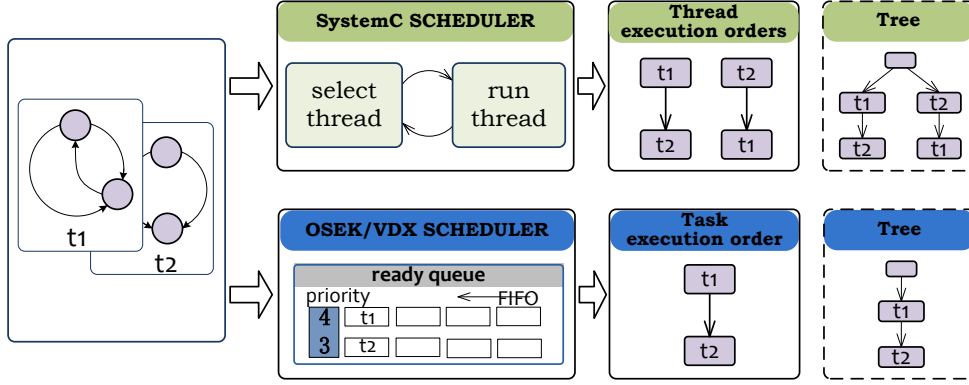


Figure 1.2: The difference of schedulers

determined by scheduler (non-deterministic scheduler is used to dispatch threads). E.g., there are two threads t_1 and t_2 in Fig. 1.2. If SystemC scheduler is used to dispatch these two threads, there are two possible thread execution orders (t_1, t_2) and (t_2, t_1) . However, compared with the non-deterministic scheduler, in OSEK/VDX applications which task is to be run is explicitly determined by OSEK/VDX scheduler—a deterministic scheduler is used to dispatch tasks. E.g., for the same example shown in Fig. 1.2, if the OSEK/VDX scheduler is used to dispatch tasks t_1 and t_2 , there is just one task execution order (t_1, t_2) , where we assume that the priority of task t_1 is higher than task t_2 and these two tasks are currently in the *ready* state.

1.3 Thesis Purpose

The purpose of this thesis is to make model checking technique more accurate in the verification of OSEK/VDX applications. In order to achieve this purpose, in this thesis we describe and develop three approaches that can accurately and automatically verify the safety property of OSEK/VDX applications using model checking technique. To the best of our knowledge, our work is first to apply model checking technique to accurately verify the multi-tasks/threads software which is dispatched by a deterministic scheduler. There are two objects in this thesis, one is to show the details of our approaches, and the other is to investigate the accuracy, efficiency and scalability of proposed approaches based on

the experiments.

1.4 Checking Approaches for OSEK/VDX Applications

In OSEK/VDX application, the running task is explicitly determined by OSEK/VDX scheduler and the APIs invoked from tasks will haphazardly change the scheduling of tasks. There are several challenges that should be addressed when we apply model checking technique to accurately verify OSEK/VDX applications, e.g.,

- How to construct an accurate checking model for OSEK/VDX applications.
- How to handle the behaviours of OSEK/VDX OS, i.e., how to deal with the APIs invoked from tasks, and how to explicitly perform the scheduling behaviours.

In our first approach, as to apply existing model checkers to accurately verify OSEK/VDX applications, we investigate a method based on the well-known Spin model checker [26]. To accurately verify OSEK/VDX applications using Spin, the key work is how to construct a checking model to describe the executions of an application. In our Spin-based checking approach, a combination model of application model and OSEK/VDX OS model is used to precisely simulate the executions of the OSEK/VDX applications in real OSEK/VDX OS, where all of the tasks within application are designed as **processes**, and the OS model that conforms to OSEK/VDX specification as a cooperative **process** is used to dispatch tasks and respond to the APIs invoked from tasks. We have conducted many experiments using this approach. The experiment results indicate that the Spin-based checking approach can accurately verify OSEK/VDX applications. However, the scalability of the approach is limited, because too many details about OS model are explored in the verification stage, especially the state space explosion [4] may happen if the checked application invokes a lot of service APIs.

In order to avoid the behaviors of OSEK/VDX OS model to be explored in the verification stage and intend to handle a complex OSEK/VDX application which holds a large

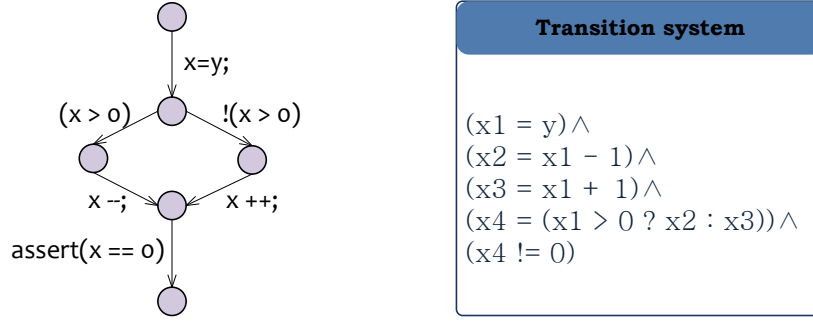


Figure 1.3: Example for bounded model checking

number of states, in our second approach we propose a new technique named execution path generator (EPG) based on the advanced SMT-based bounded model checking (BMC) [11][14]. The core work of verifying a system using BMC is how to construct a transition system (checking model) to reflect the executions of given system. As shown in Fig. 1.3, we can use the transition system to reflect the executions of the example. However, compared with the simple example shown in Fig. 1.3, an OSEK/VDX application holds multiple tasks and a scheduler is used to dispatch tasks. Moreover, the APIs invoked from tasks will haphazardly change the scheduling of tasks, e.g., the different APIs locating at different branches will lead to the different task execution orders. In contrast with the shown example, it is difficult to construct a transition system for OSEK/VDX application.

In our EPG technique, like other BMC methods for concurrent software (e.g., the method [37] for ANSI-C multi-threaded software), we explore all of the execution paths for constructing the transition system in order to successfully apply BMC technique to verify OSEK/VDX applications. Particularly, as to avoid the behaviors of OSEK/VDX OS model to be explored in the verification stage and construct an accurate transition system, the OSEK/VDX OS model is embedded in EPG (the algorithm of constructing model flat) to dispatch tasks and respond to the APIs invoked from tasks. There are two advantages in our EPG technique, e.g., (i) EPG: we can construct an accurate transition system without the behaviors of OSEK/VDX OS model for OSEK/VDX application based on the generated execution paths, since OSEK/VDX OS model is embedded in EPG to dispatch tasks and respond to the APIs invoked from tasks. (ii) SMT-based BMC: it can

make our approach more efficient and scalable in checking the complex applications which hold a large number of states, because the state-of-the-art SMT solver [10][17] is used to carry out the verification.

We have developed a corresponding tool named **osek-bmc** according to the proposed approach, and conducted many experiments using the developed tool. The experiment results show that, our EPG technique is more efficient and scalable than Spin-based checking approach in checking the OSEK/VDX applications which hold a lot of tasks and APIs, but it is not efficient to handle the applications which hold a lot of branches. This is because, in our EPG technique the transition system of an OSEK/VDX application is constructed based on the execution paths. If the application holds a lot of branches, it will spend a lot of time exploring execution paths for constructing transition system, which will slow down the performance of our EPG technique.

As to efficiently check the large-scale applications which hold a lot of tasks, APIs and branches, and moreover, enable existing model checkers such as **spin** [26] and **cbmc** [16] to directly verify OSEK/VDX applications, in our third approach we propose a novel method to translate OSEK/VDX applications into sequential models based on the sequentialization idea. In order to efficiently and accurately sequentialize OSEK/VDX applications, There are several challenges that should be addressed, e.g.,

- How to explicitly perform the scheduling behaviours of OSEK/VDX OS and deal with the invoked APIs.
- How to efficiently construct a sequential model to describe the behaviours of an OSEK/VDX application, e.g., how to efficiently handle branches and the same execution behaviours of application.

In our sequentialization-based approach, like our EPG technique, we embed OSEK/VDX OS model in the translation algorithm flat to explicitly perform the scheduling behaviours and respond to the APIs invoked from tasks. In addition, as to efficiently construct a sequential model, an extended directed graph is used to implement the sequentialization process of OSEK/VDX application. The key idea of sequentialization-based approach is

that, we symbolically execute application using the extended directed graph and explicitly perform the scheduling behaviours. In other words, in the sequentialization process task statements such as condition statements and assignment statements are not computed, instead, the extended directed graph is used to describe the executions of application under the dispatch of embedded OSEK/VDX OS model. There are several advantages in our sequentialization-based approach, e.g.,

- It can efficiently construct a sequential model for OSEK/VDX applications based on the directed graph, e.g., we can employ the combination states in directed graph to avoid the process of exploring execution paths, and use cycles to represent the same execution behaviours such as loops in application.
- The constructed sequential model does not hold the behaviours of OSEK/VDX OS model, because OSEK/VDX OS model is embedded in translation algorithm flat to dispatch tasks and respond to the invoked APIs.
- Based on the sequentialization process, the existing model checkers can be directly applied to verify OSEK/VDX applications.

We have implemented a tool named `autoC` according to the sequentialization approach, and conducted many experiments using the implemented tool. In the experiments, to comprehensively investigate the effectiveness of our sequentialization-based approach, the OSEK/VDX applications which hold different task number, API number, loop number and different scheduling behaviours are selected as our benchmarks. In addition, in the experiments, the well-known Spin is selected as the back-end model checker, and we compared our sequentialization-based approach with Spin-based checking approach and EPG technique. According to the conducted experiment results, we find the following results,

- (i) The Spin-based checking approach can accurately verify the OSEK/VDX applications, but the scalability of this approach is limited because too many details about

OSEK/VDX OS model are explored in the verification stage, especially the state space explosion will happen if the checked applications hold a lot of tasks and APIs.

- (ii) EPG technique is more efficient and scalable than Spin-based checking approach in checking the applications which hold a lot of tasks and APIs. However, it is not efficient to check the applications which hold a large number of branches, since the transition system of application is constructed based on the execution paths in EPG technique. If the application holds a lot of branches, it will spend a lot of time exploring execution paths for constructing transition system, which will slow down the performance of our EPG technique.
- (iii) Based on the sequentialization process of our approach, the selected back-end model checker Spin can efficiently verify the applications which hold a lot of tasks, loops and APIs with the less cost in terms of states, time and memory compared with the Spin-based checking approach and EPG technique.

Furthermore, we have used our sequentialization-based approach to sequentialize many experimental OSEK/VDX applications which hold about 1000 lines of C code, and verified the sequentialized applications with the well-known bounded model checker `cbmc`. The performances indicate that the sequentialization-based approach and `cbmc` can be considered as a practical method to verify the OSEK/VDX applications with industrial complexity.

1.5 Thesis Contribution

With the growing functionalities in vehicles and increasing complexity in the development, how to straitly ensure the reliability of the developed OSEK/VDX applications is becoming a challenge for developers. The approaches presented in this thesis can help developers to accurately check the developed OSEK/VDX applications. There are two main contributions in this thesis. The first contribution is, our approaches can reduce the

costs on the verification of OSEK/VDX applications, which benefits from the following superiorities,

1. Developers can easily use our approaches to verify an OSEK/VDX application, since our approaches support all of the OSEK/VDX OS modules and related APIs. Particularly, the corresponding tools support a subset of C programming language as input, developers can use the corresponding tools to automatically carry out the verification.
2. In the verification process, developers do not need to judge whether the detected bug is a real one or not after completing verification. This is because, the behaviours of OSEK/VDX scheduler are taken into account in our approaches. Based on the scheduling of OSEK/VDX scheduler, our approaches can perform an accurate verification.
3. Our approaches make the checking process of OSEK/VDX applications more trustworthy, since our approaches are based on the model checking that is an exhaustive checking technique.
4. Our approaches will return a counterexample when finding a bug. Based on the returned counterexample, developers can easily understand the bug and correct the bug.

The second contribution comes from our sequentialization-based checking approach. Based on the sequentialization process of our approach,

1. Developers can verify a large-scale application using existing model checkers, since the existing model checkers just verify a sequential model instead of a concurrent model.
2. Developers can conveniently select suitable model checkers to verify OSEK/VDX applications, e.g., use `cbmc` to efficiently find subtle bugs, and use `esbmc` [31] to verify the applications which hold unbounded loops.

1.6 Thesis Outline

The rest of the thesis is structured as follows. The background for model checking technique is presented in chapter 2. As to easily understand our checking approaches, the preliminaries for OSEK/VDX OS and applications are discussed in chapter 3. Based on the discussion about the execution characteristics of OSEK/VDX applications, the overviews of our three checking approaches are stated in chapter 4. Then, the details of our approaches are presented in chapter 5 to chapter 7. As to show the accuracy, efficiency and scalability of our approaches, the experiments are carried out in chapter 8. Related work is discussed in chapter 9. Conclusion of thesis is placed in the last chapter.